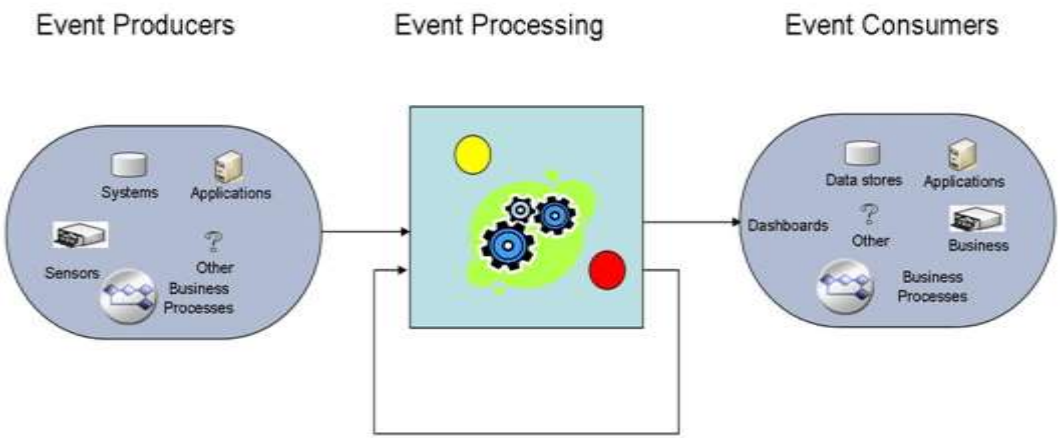
Lecture 2
Architectural Styles
Part-II

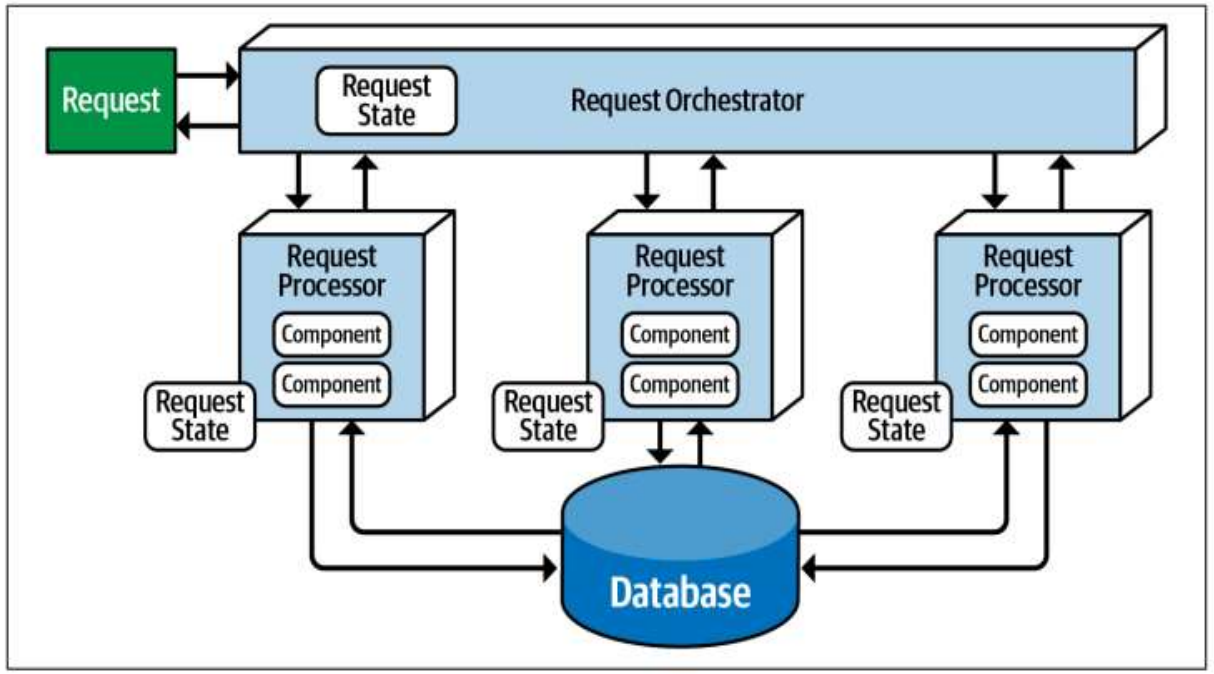
5. Event driven architecture



- Popular distributed asynchronous architecture style used to produce highly scalable and high-performance applications.
- It is also highly adaptable and can be used for small applications and as well as large.
- It is made up of decoupled event processing components that asynchronously receive and process events.
- It can be used as a standalone architecture style or embedded within other architecture styles (such as an event driven microservices architecture).

5. Event driven architecture

- Mainly applications follow request-based model which requests are send to a request orchestrator (a user interface implemented through an API layer or enterprise service bus).
- The request orchestrator deterministically and synchronously direct requests to various request processors.
- The request processors handle the request, either retrieving or updating information in a database.



Request-based model

5. Event driven architecture

Topology

- Two primary topologies within event-driven architecture:
    - *Mediator topology*
    - *Broker topology*
- The **mediator topology** is commonly used when you require control over the workflow of an event process, whereas,
- The **broker topology** is used when you require a high degree of responsiveness and dynamic control over the processing of an event.
- The architecture characteristics and implementation strategies differ between these two topologies

5. Event driven architecture

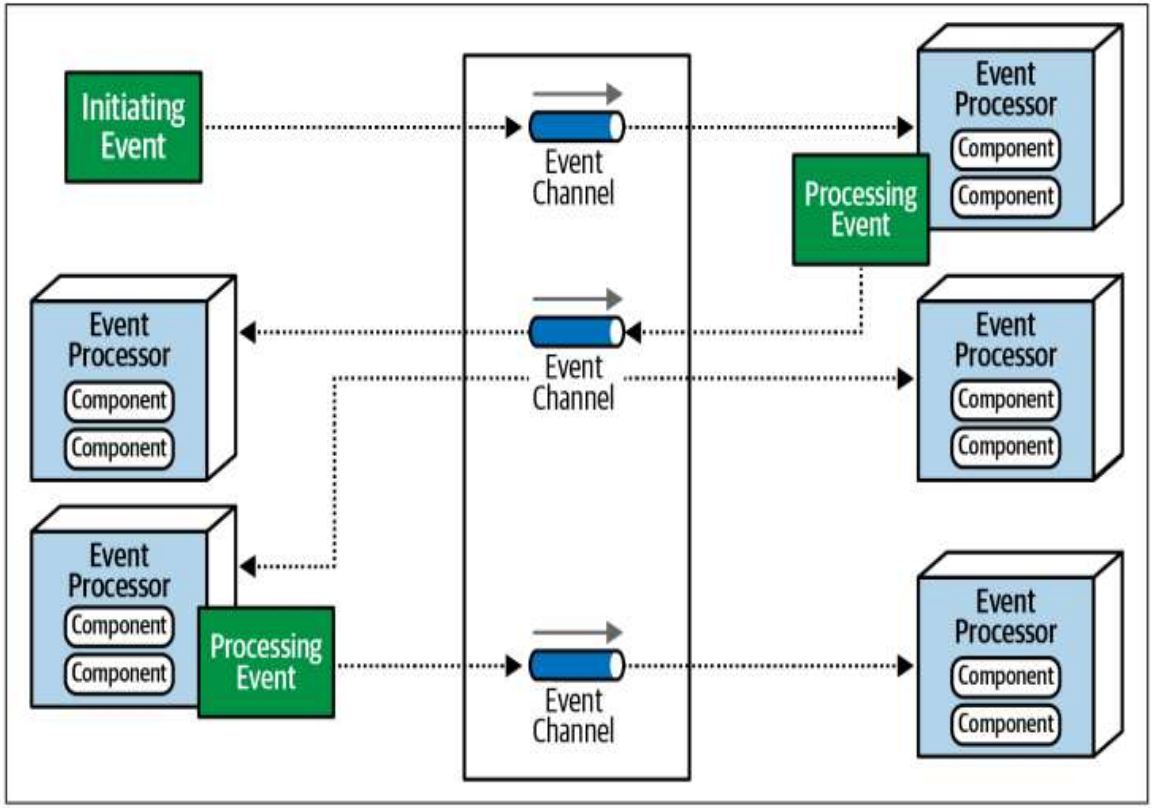
Broker topology

- Differs from the mediator topology in that there is no central event mediator
- The message flow is distributed across the event processor components in a chain-like broadcasting fashion
- This topology is useful do not need central event orchestration and coordination
- There are four primary architecture components within the broker topology:
  - An initiating event
  - The event broker
  - An event processor
  - A processing event

5. Event driven architecture

Broker topology…

- The initial event that starts the entire event flow and sent to an event channel in the event broker for processing
- The event processor that accepted the initiating event from the event broker performs a specific task associated with the event
- Then the event processor asynchronously advertises what it did to the rest of the system by creating what is called a processing event.
- Other event processors listen to the processing event, react to that event by doing something
- Topics are usually used in the broker topology using a *publish-and-subscribe messaging model*

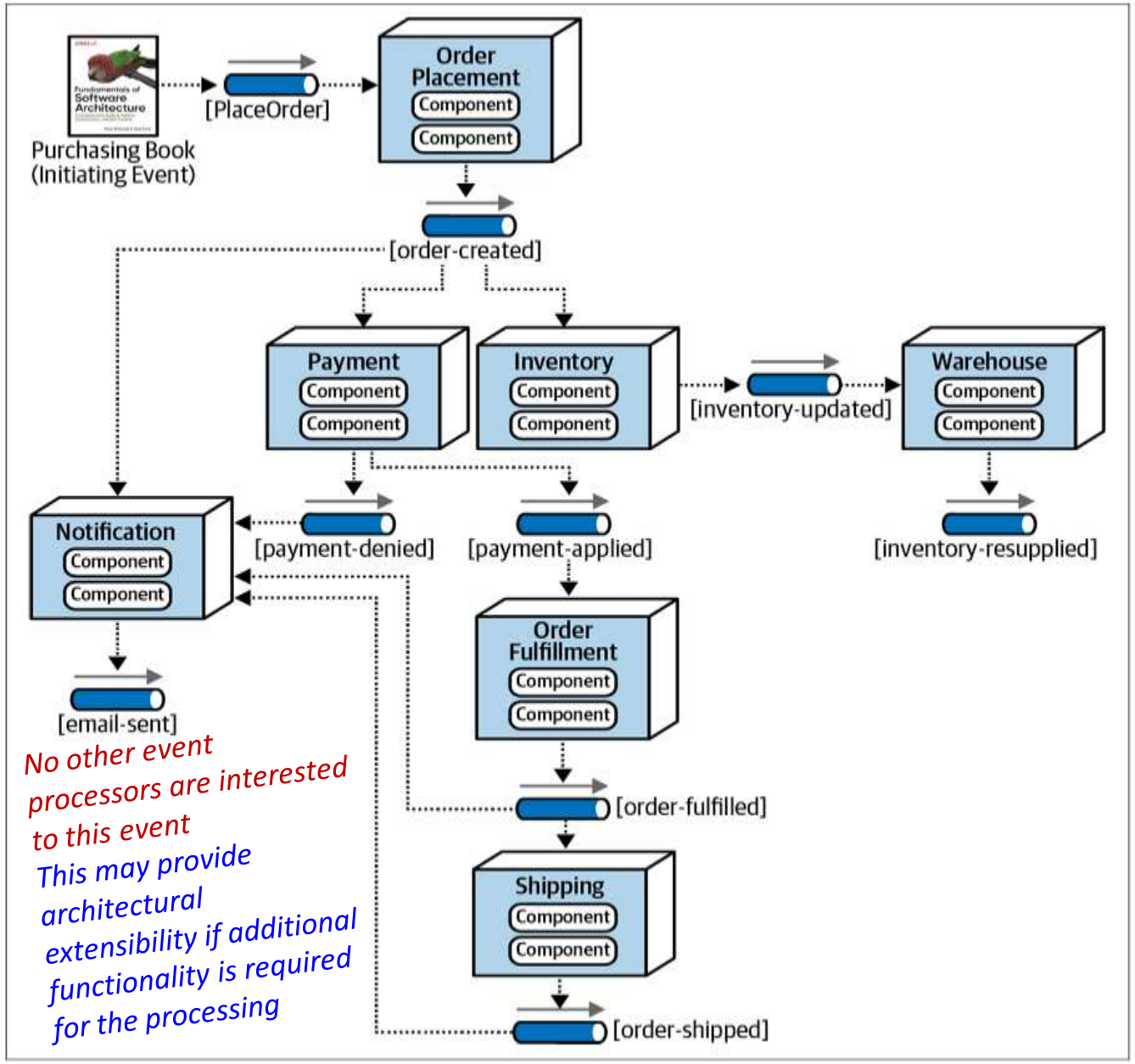


Broker topology

5. Event driven architecture

Retail order entry system example

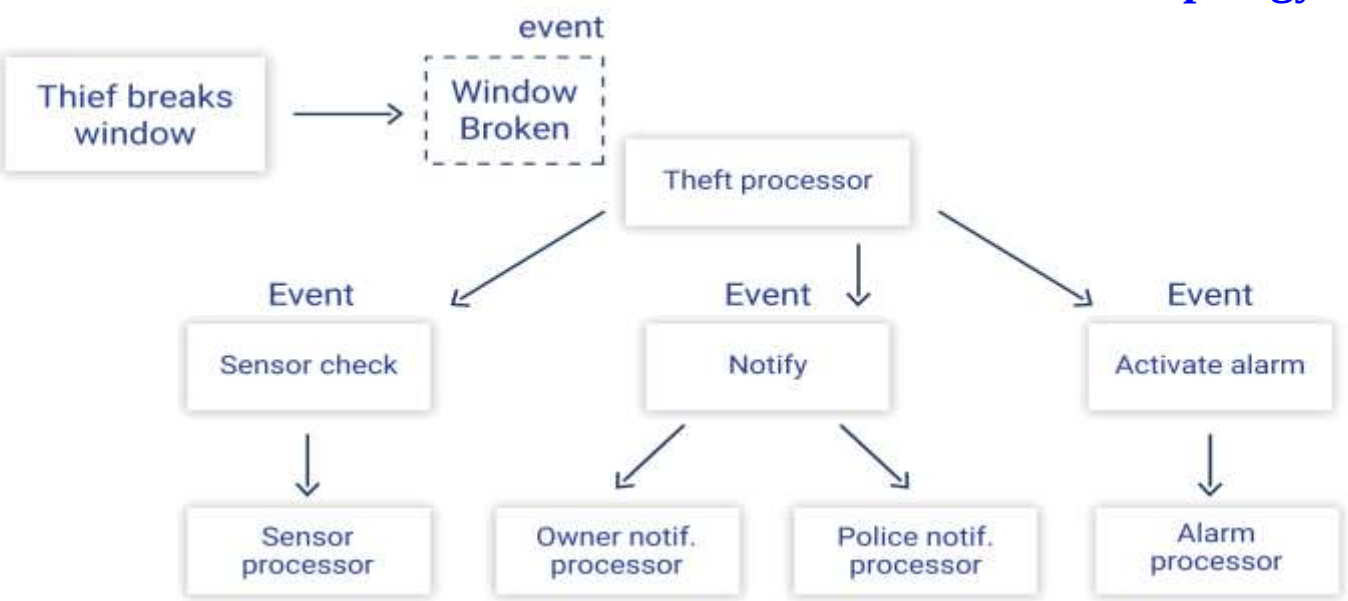
- An Initiating Event create an event called PlaceOrder
- The OrderPlacement event processor receives the initiating event (PlaceOrder), then advertises to the rest of the system that it created an order through an order-created processing event
- Three event processors are interested in that event: the Notification event processor, the Payment event processor, and the Inventory event processor.
    - All three of these event processors perform their tasks in parallel and create an event for further processing.
- Notice that all of the event processors are highly decoupled and independent of each other



5. Event driven architecture

Broker topology….

- A jewelry store example, a breach of any of the security devices, such as breaking a window, creates an event that is sent directly to the broker.
- The broker communicates the event notification directly to the event processors. Since nothing is asynchronous, **no event queue is required**.
- At the end of each process, notification of completion is issued by the event processors.



Advantages	Disadvantages
Highly decoupled event processors	Workflow control
High scalability	Error handling
High responsiveness	Recoverability
High performance	Restart capabilities
High fault tolerance	Data inconsistency

Trade-off of the broker topology

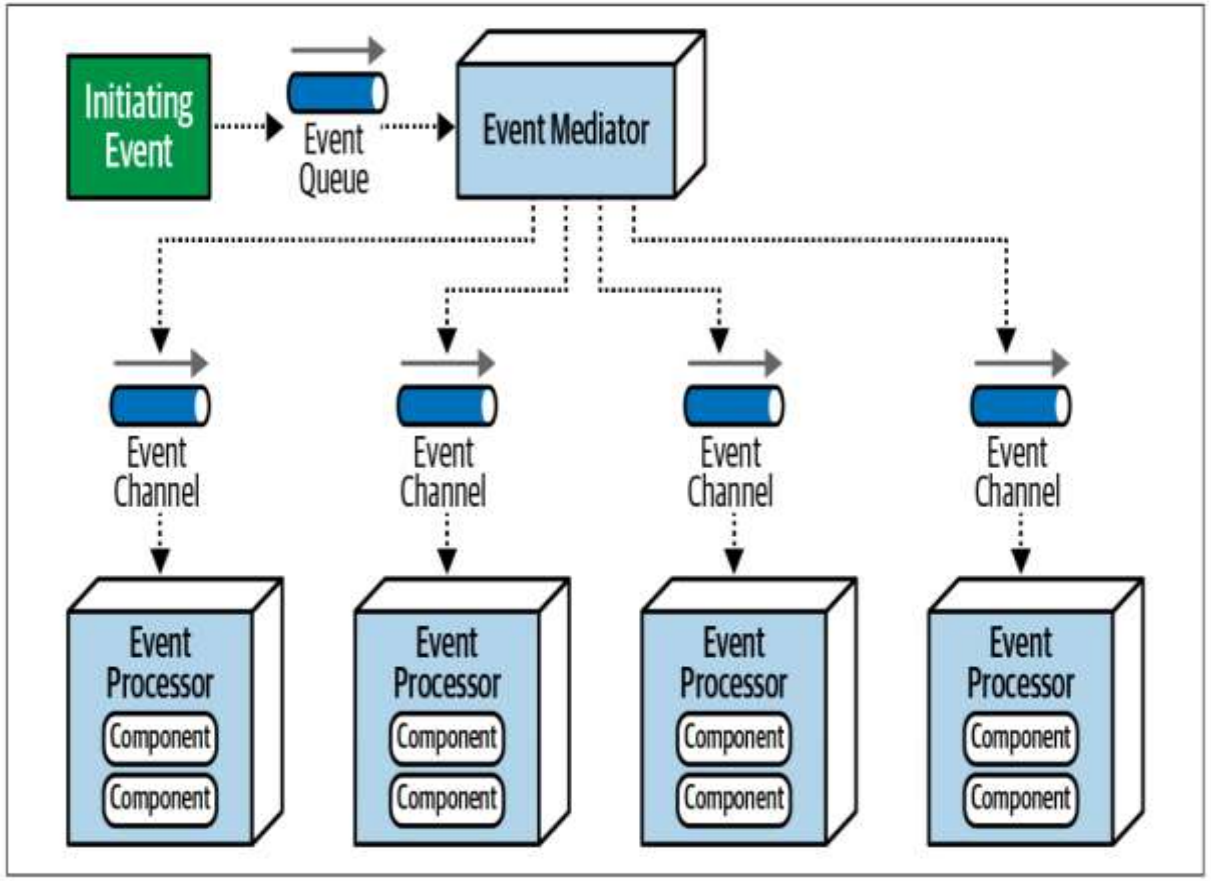
5. Event driven architecture

Mediator Topology

- Manages and controls the workflow for initiating events that require the coordination of multiple event processors.
- The architecture components that make up the mediator topology are:
  - An initiating event— starts the whole evening process
  - An event queue — store events before sent to the mediator,
  - An event mediator — passes events to the event processing,
  - An event channels — a routing node in the event processing and
  - An event processors — process the event, and respond completion back to the mediator.

Mediator Topology

- The event mediator generates corresponding processing events that are sent to the event channels in a (queue) point-to-point messaging fashion.
- An event processors within the mediator topology do not advertise what they did to the rest of the system, usually respond back to the mediator
- There are multiple mediators, associated with a particular domain or grouping of events.
- This reduces the single point of failure and increases overall throughput and performance.



Mediator topology

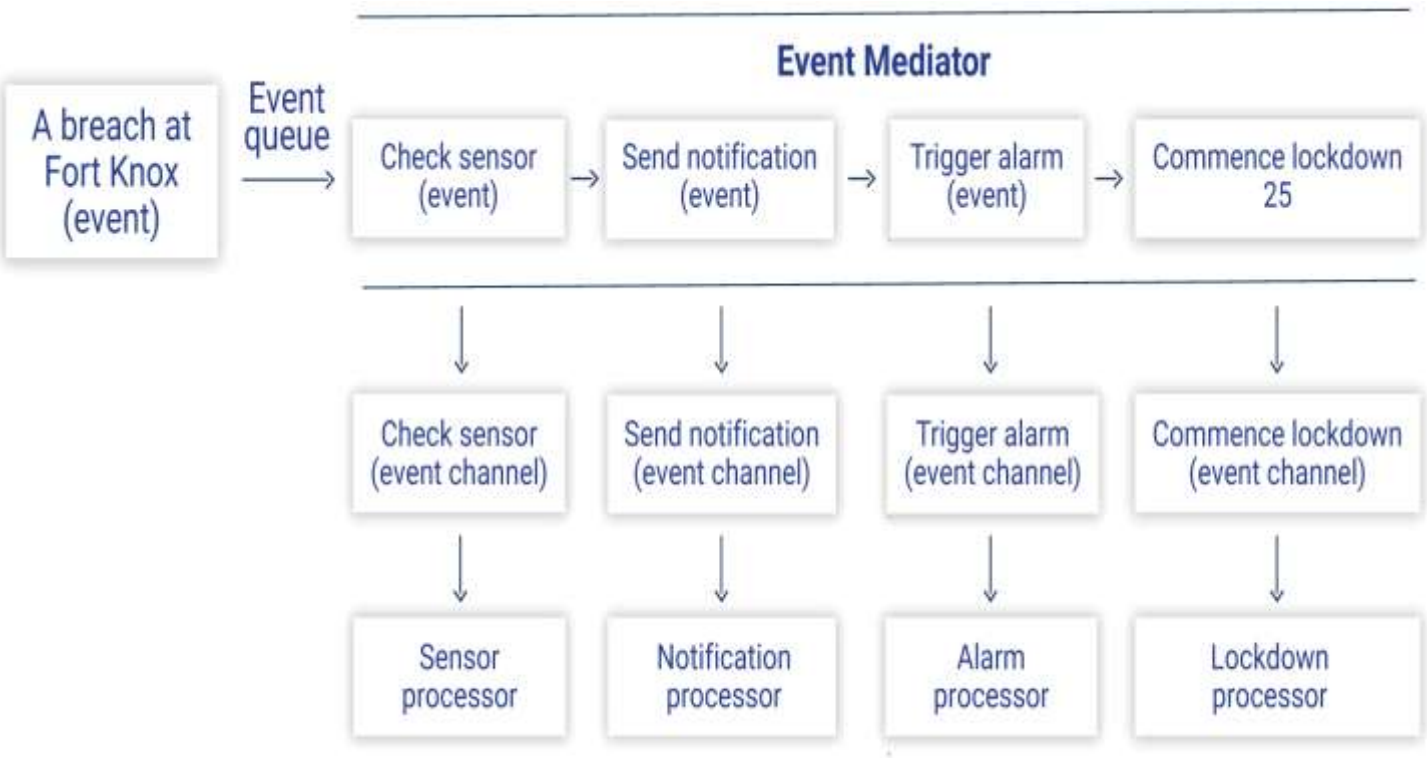
Mediator Topology

Advantages	Disadvantages
Workflow control	More coupling of event processors
Error handling	Lower scalability
Recoverability	Lower performance
Restart capabilities	Lower fault tolerance
Better data consistency	Modeling complex workflows

- The choice between the broker and mediator topology essentially comes down to a trade-off between workflow control and error handling capability versus high performance and scalability.

Architecture Styles

Mediator Topology



- For example, at Fort Knox, many things must happen instantly and some later on. These include a complex series of time-critical events such as **alerts to enforcement personnel, shutting of emergency doors, lights coming on, and warning siren**.

Architecture Characteristics Ratings

Multiple event processors and tied together through mediators, queues, and topics

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★
Evolutionary	★★★★★
Fault tolerance	★★★★★
Modularity	★★★★
Overall cost	★★★
Performance	★★★★★
Reliability	★★★
Scalability	★★★★★
Simplicity	★
Testability	★★

6. Space-Based Architecture Style

- With increased users, scaling the web server, then scaling app server and finally scaling the DB server can minimize the performance degrade.
- This is relatively easy and inexpensive, However, in most cases of high user load, scaling out the web-server layer just moves the bottleneck down to the application server.
- Scaling application servers further moves the bottleneck down to the database server, which is even more difficult and expensive to scale creating triangular shaped topology, with the widest part of the triangle being the web servers (easiest to scale).

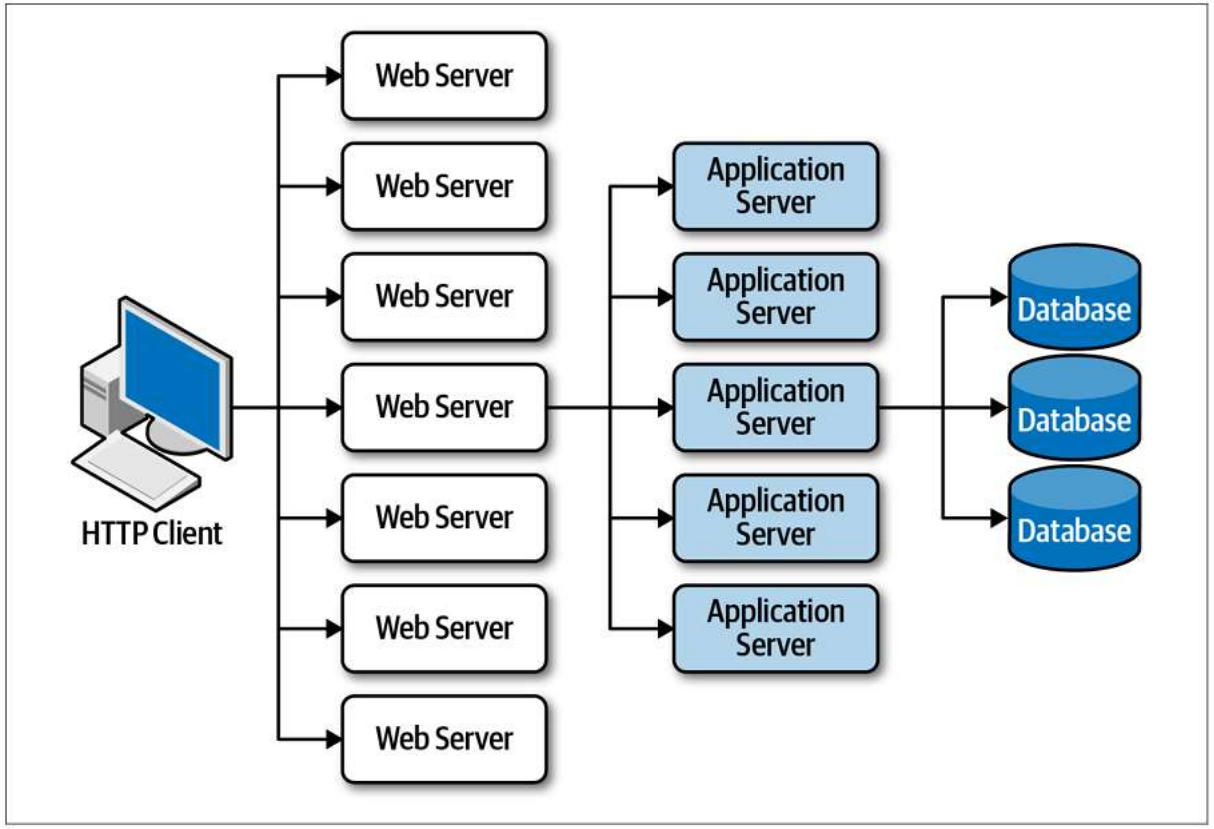
6. Space-Based Architecture Style

- Any application with a large concurrent user load, concurrent transactions limits database access speed.
- Caching and database scaling products help to address these issues, the fact remains that scaling out a normal application for extreme loads is a very difficult.
- Space-based architecture style is specifically designed to address problems involving high scalability, elasticity, and high concurrency issues.
- Its also useful for applications that have variable and unpredictable concurrent user volumes.

6. Space-Based Architecture Style

Even if we can scale the database, what we eventually end up with is a triangle shaped topology,

- with the widest part of the triangle being the web servers (easiest to scale) and
- the smallest part being the database (hardest to scale),



Scalability limits within a traditional web-based topology

6. Space-Based Architecture Style

General Topology

- Space-based architecture leverage the technique of using multiple parallel processors communicating through shared memory
- High scalability, high elasticity, and high performance are achieved by removing the central database as a synchronous constraint in the system and instead leveraging replicated in-memory data grids.
- Application data is kept in-memory and replicated among all the active processing units. When a processing unit updates data, it asynchronously sends that data to the database, usually via messaging with persistent queues.
- Processing units start up and shut down dynamically as user load increases and decreases, thereby addressing variable scalability.
- There is no central database involved in the application, the database bottleneck is removed, thus providing near-infinite scalability within the application.

6. Space-Based Architecture Style

There are several architecture components that make up a space-based architecture:

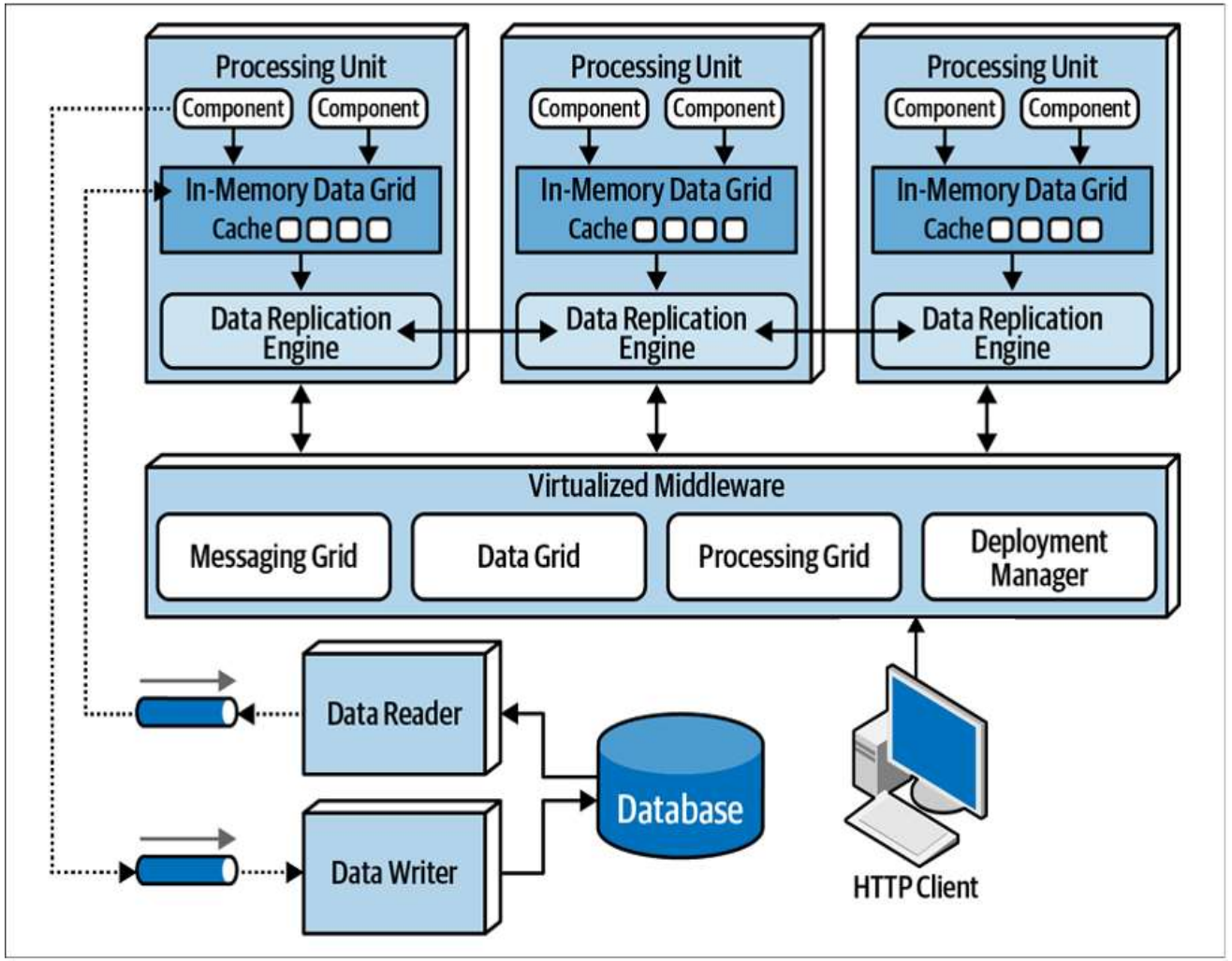
- processing unit containing the application code,
- virtualized middleware used to manage and coordinate the processing units,
- data pumps to asynchronously send updated data to the database,
- data writers that perform the updates from the data pumps, and
- data readers that read database data and deliver it to processing units upon startup

Smaller web-based applications would likely be deployed into a *single processing unit*, whereas *larger applications* may split the application functionality into multiple processing units

6. Space-Based Architecture Style

Processing Unit

- The processing unit contains the application logic (web-based component and back-end business logic)
- Depending on the size of the functionality processing units may deploy all or part of the business logic.



Space-based architecture basic topology

6. Space-Based Architecture Style

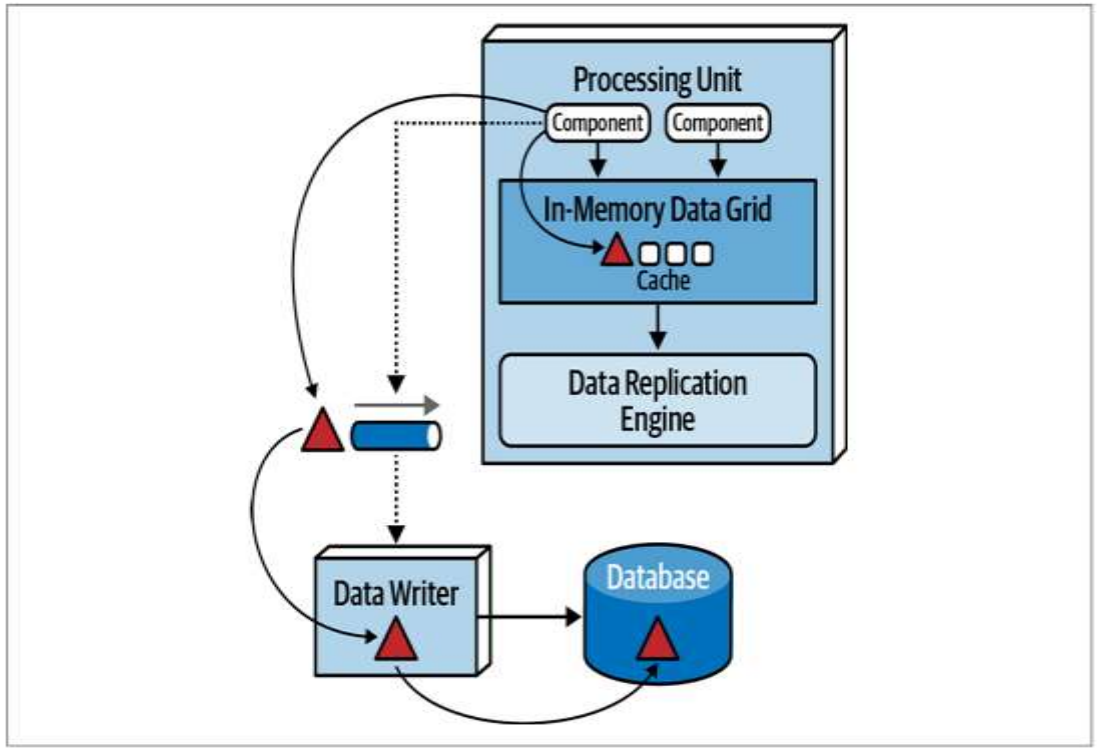
Virtualized Middleware

- The virtualized middleware handles the infrastructure concerns within the architecture that control various aspects of data synchronization and request handling.
- The components in virtualized middleware include
    - Messaging grid- manages input request and session state. Determines which active processing components are available to receive the request
    - Data grid -maintains the data across processing caches , Data is synchronized between processing units that contain the same named data grid
    - Processing grid- manages orchestrated request processing and
    - Deployment manager -manages the dynamic startup and shutdown of processing unit instances based on load conditions. starts up new processing units when load increases, and shuts down processing units when the load decreases

6. Space-Based Architecture Style

Data Pumps

- Manages data as processing units do not directly read from and write to a database.
- Always asynchronous, providing eventual consistency with the in-memory cache and the database.
- Usually implemented using messaging supporting asynchronous communication, guaranteed delivery, and preserving message order via FIFO.
- Messaging provides a decoupling b/n processing units and data writer, if the data writer is not available, uninterrupted processing can still take place within the processing units

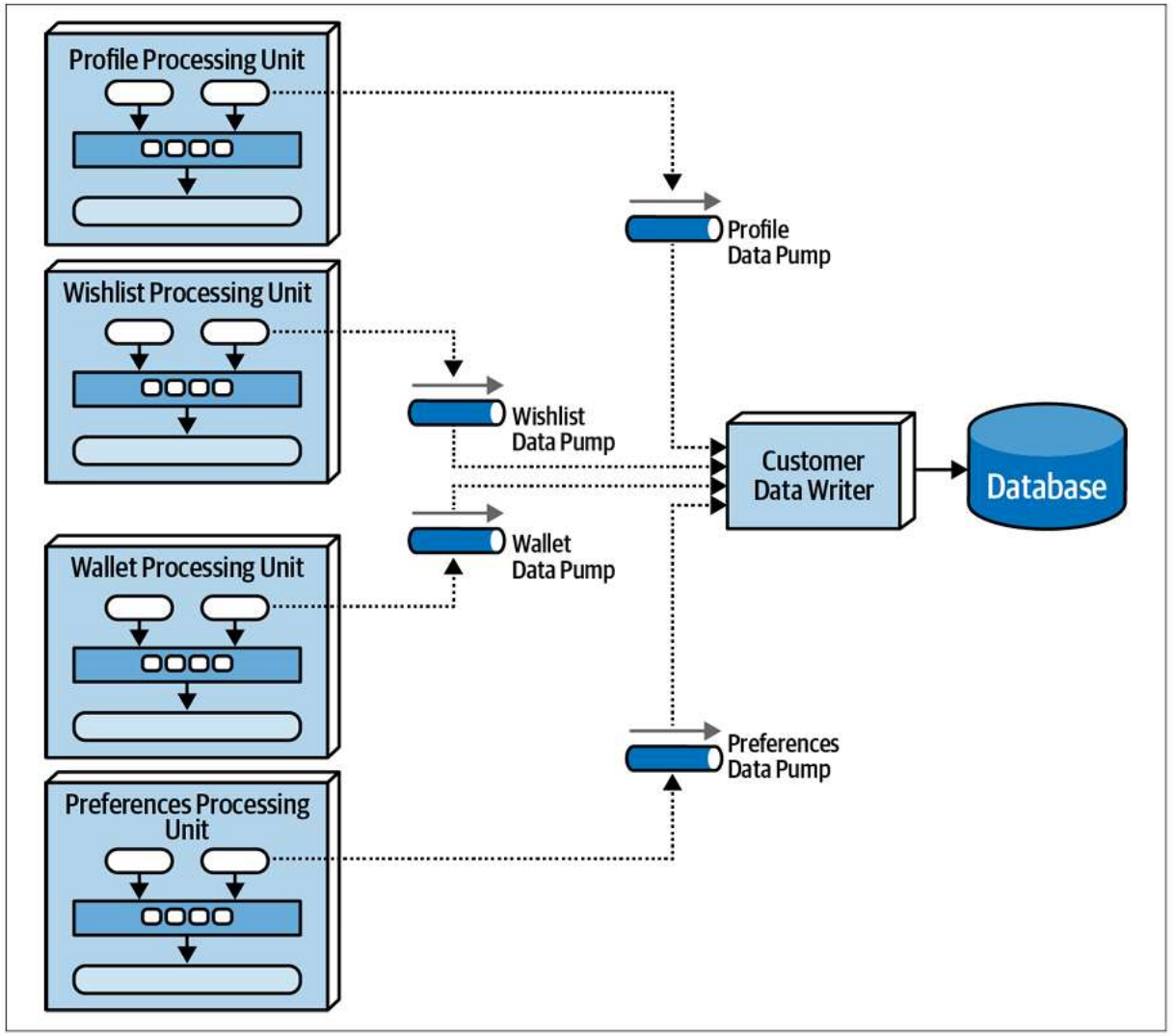


Data pump used to send data to a database

6. Space-Based Architecture Style

Data Writers

- The data writer component accepts messages from a data pump and updates the database.
- Data writers can be implemented as services, applications, or data hubs (such as Ab Initio).
- Notice there are four different processing units and data pumps representing the customer domain but only one data writer.
- The single data writer listens to all four data pumps.

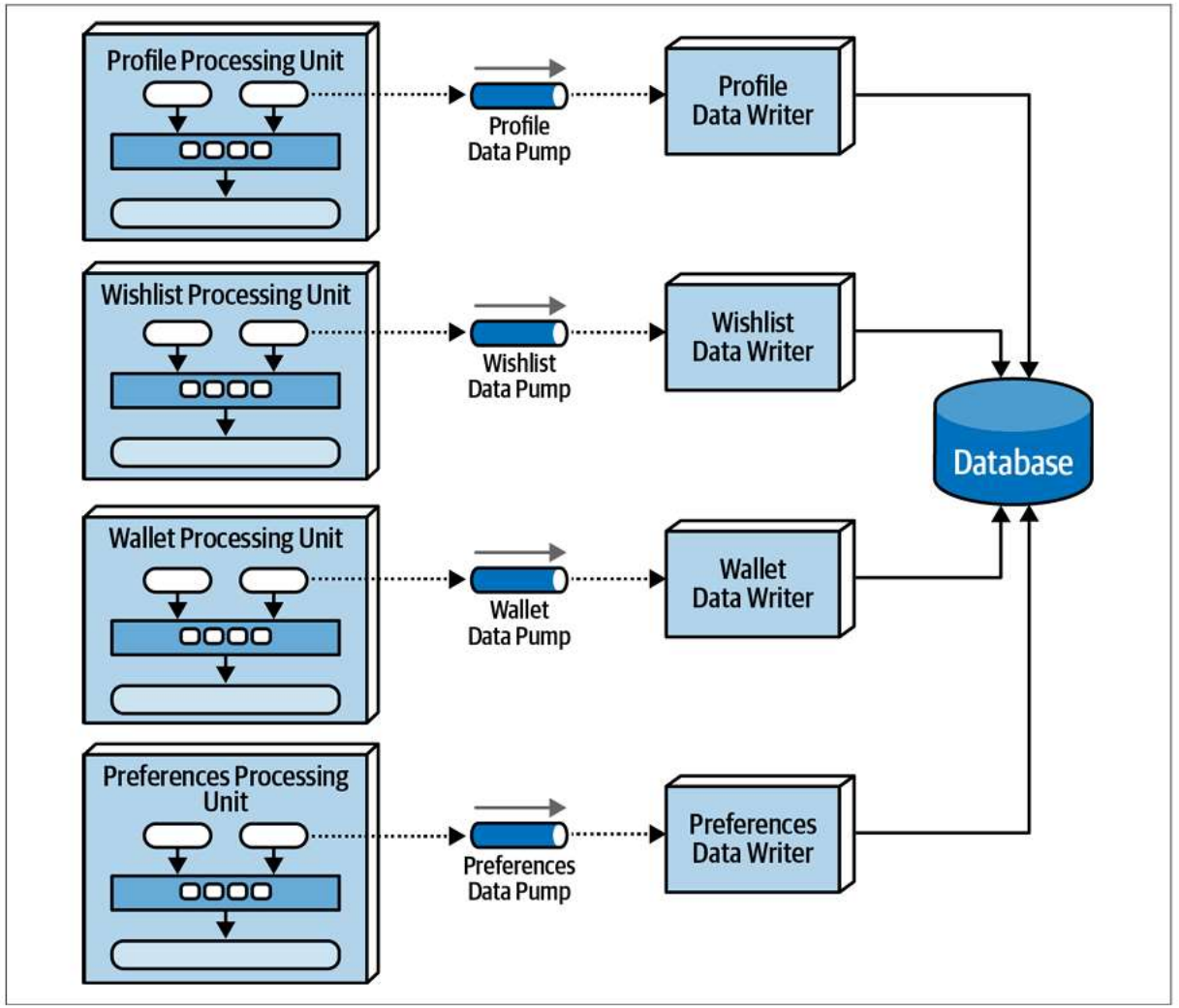


Domain-based data writer

6. Space-Based Architecture Style

Data Writers

- Alternatively, each class of processing unit can have its own dedicated data writer component.
- In this model the data writer is dedicated to each corresponding data pump.
- While this model tends to produce too many data writer components, it does provide better scalability and agility due to the alignment of processing unit, data pump, and data writer

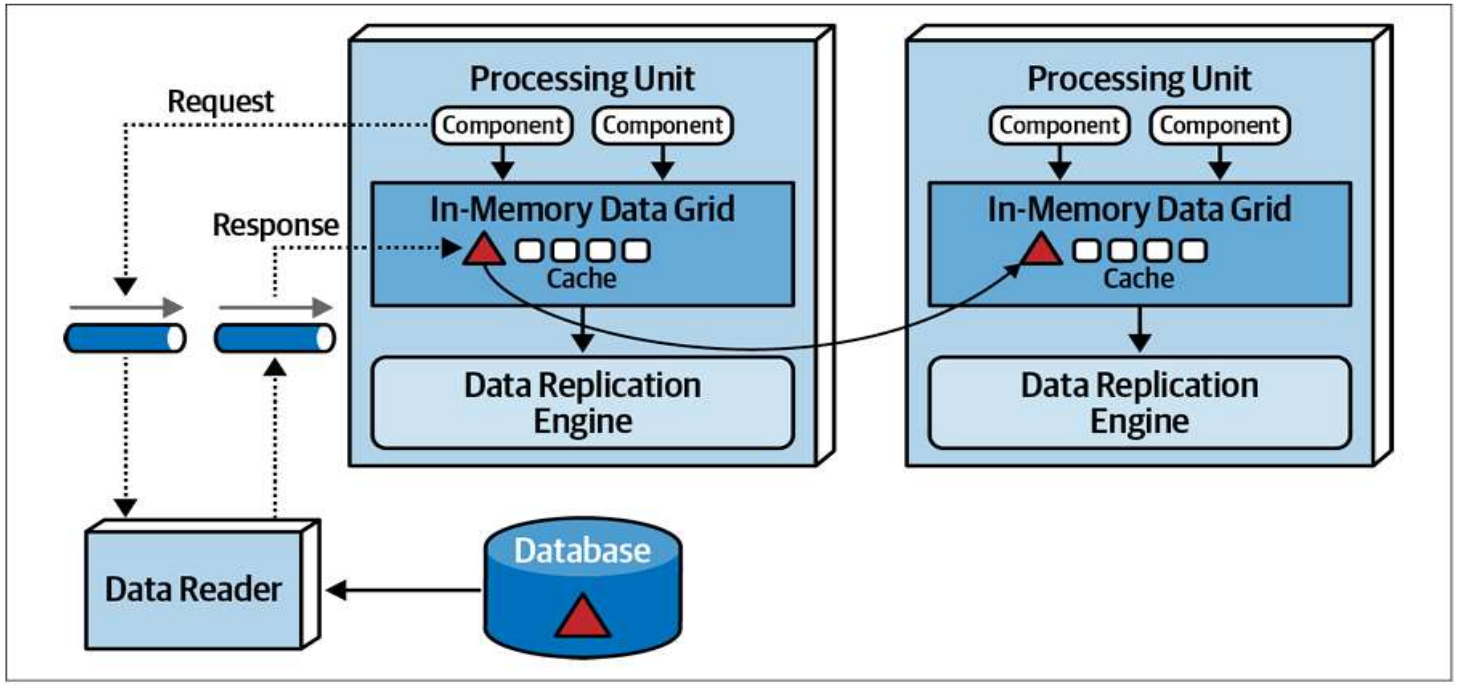


Dedicated data writers for each data pump

6. Space-Based Architecture Style

Data Readers

- Data readers take on the responsibility for reading data from the database and sending it to the processing units via a reverse data pump



- Data readers are only invoked under one of three situations:
  - A crash of all processing unit instances of the same named cache
  - A redeployment of all processing units within the same named cache, or
  - Retrieving archive data not contained in the replicated cache

6. Space-Based Architecture Style

Examples of applications that can benefit from Space-Based Architecture:

- E-commerce platforms: To handle high volumes of traffic and transactions
- Online gaming platforms: To handle real-time game events and player interactions
- Financial trading systems: Handle high volumes of transactions and market data
- Telecommunications systems: Handle high volumes of call traffic and messaging
- Healthcare applications: to handle large volumes of patient data and medical records
- Overall, Space-Based Architecture is well-suited for applications that require high scalability, high performance, and fault tolerance by distributing the workload across multiple nodes..

6. Space-Based Architecture Style

Implementation Examples

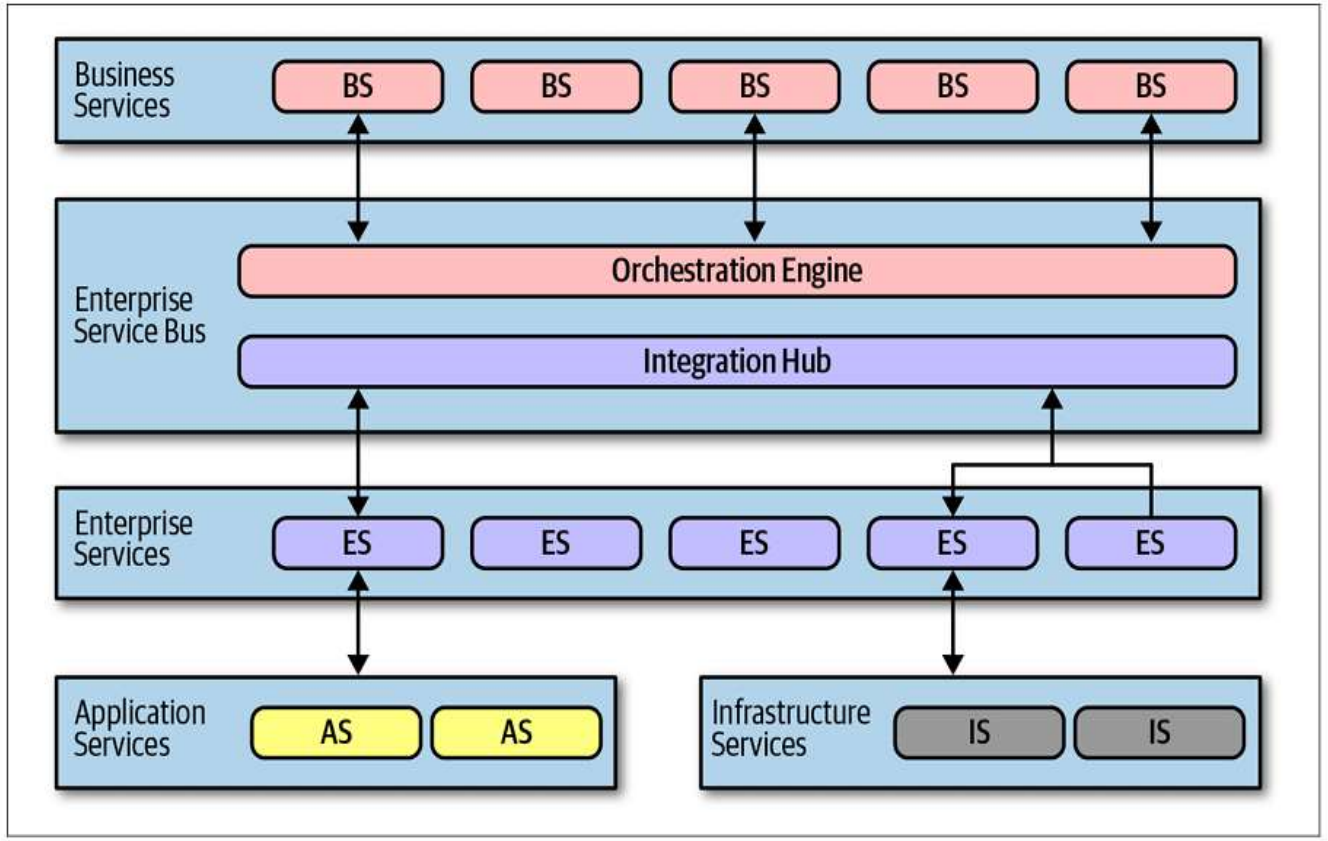
- Space-based architecture is well suited for applications that experience high spikes in user or request volume and applications that have throughput in excess of 10,000 concurrent users.
- Examples of space-based architecture include applications like
    - online concert ticketing systems and
    - online auction systems.
- Both of these examples require high performance, high scalability, and high levels of elasticity

Architecture characteristics ratings

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★★★
Evolutionary	★★★
Fault tolerance	★★★
Modularity	★★★
Overall cost	★★
Performance	★★★★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★

7. Orchestration-Driven Service-Oriented Architecture

- Service-Oriented Architecture (SOA) is distributed architectural style.
- Applications built using the SOA style deliver functionality as services that can be used or reused when building applications
- SOA uses open standards to integrate software assets as services
- It provides a standard form of interactions of services
- SOA services become the building blocks that form business flows
- Services can be reused by other applications
- A service provides a discrete business function that operates on data.



***Topology** of orchestration-driven service-oriented architecture*

7. Orchestration-Driven Service-Oriented Architecture

Characteristics of a Service

- Supports open standards for integration: Although proprietary integration mechanisms may be offered by the SOA infrastructure, SOA's should be based on open standards.
- Open standards: ensure the broadest integration compatibility opportunities
- Loose coupling: provides well defined interfaces to clients
- Stateless: The service does not maintain state between invocations. If a transaction is involved, the transaction is committed and the data is saved to the database.
- Location agnostic: Users of the service do not need to worry about the implementation details for accessing the service. The SOA infrastructure will provide standardized access mechanisms with service-level agreements.

7. Orchestration-Driven Service-Oriented Architecture

- A service is a reusable component that can be used as a building block to form larger, more complex business-application functionality.
- A service may be as simple as “get me some person data,” or as complex as “process a disbursement.”

Business Services (BS)

- Sit at the top of this architecture and provide the entry point.
- Example, services like ExecuteTrade or PlaceOrder represent domain behavior.
- These service definitions contained no code—just input, output, and sometimes schema information

7. Orchestration-Driven Service-Oriented Architecture

Enterprise Services (ES)

- Contain fine-grained, shared implementations. Typically, a team of developers is tasked with building atomic behavior around particular business domains. E.g. CreateCustomer, CalculateQuote..
- These services are the building blocks that make up the coarse-grained business services, tied together via the orchestration engine.
- This separation of responsibility flows from the reuse goal in this architecture.
- If developers can build fine-grained enterprise services at just the correct level of granularity, the business won't have to rewrite that part of the business workflow again.

Application Services (AS)

- Application services are single-implementation services. Not a reusable service as ES

Infrastructure services (IS)

- Supply the operational concerns (Operational services), such as monitoring, logging, authentication, and authorization. owned by a shared infrastructure team

7. Orchestration-Driven Service-Oriented Architecture

Orchestration Engine

- The orchestration engine defines the relationship between the business and enterprise services, how they map together, and where transaction boundaries lie.
- It acts as an integration hub, allowing architects to integrate custom code with package and legacy software systems.

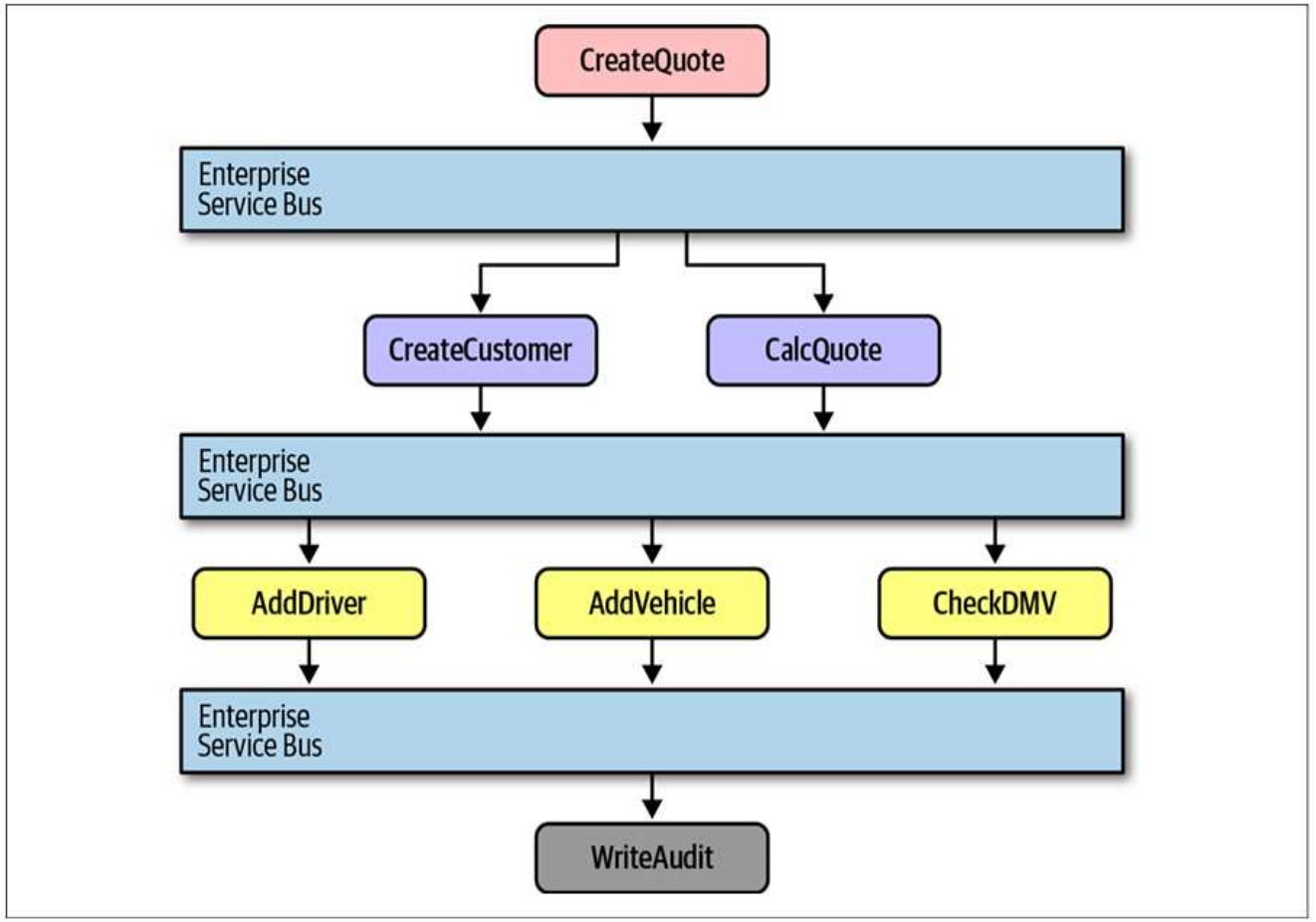
Message Flow

- Handle requests, Calls, and responses.
- All requests go through the orchestration engine—it is the location within this architecture where logic resides. Thus, message flow goes through the engine even for internal calls

7. Orchestration-Driven Service-Oriented Architecture

Message Flow

- The CreateQuote business-level service calls the service bus, which defines the workflow that consists of calls to CreateCustomer and CalculateQuote, each of which also has calls to application services.
- The service bus acts as the inter-mediary for all calls within this architecture, serving as both an integration hub and orchestration engine

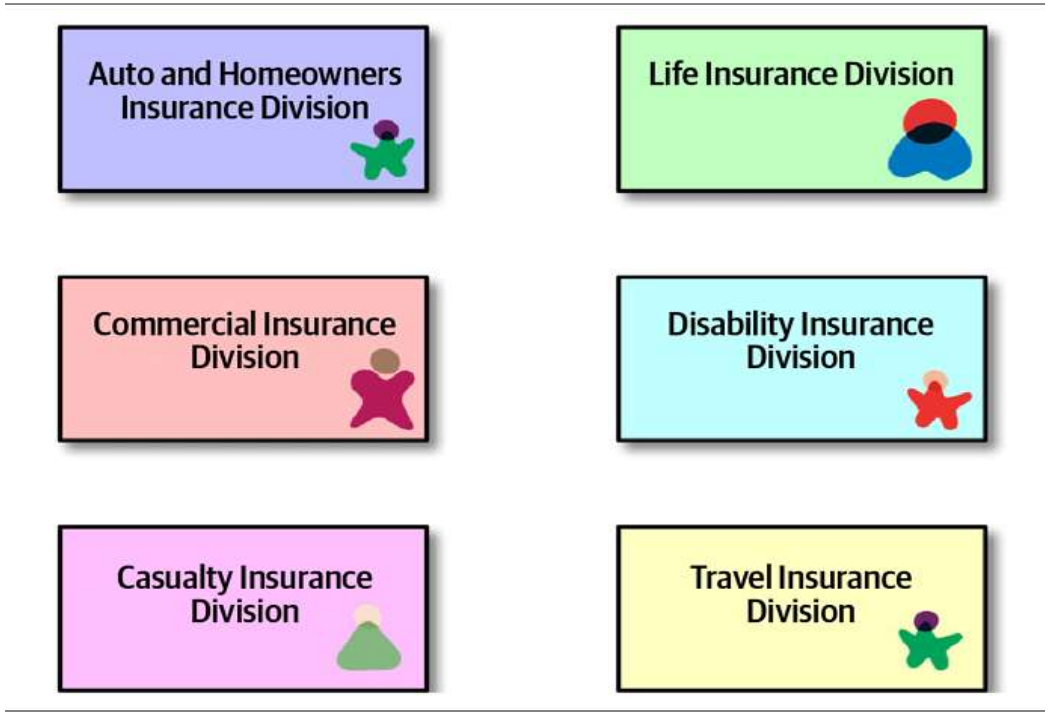


Message flow with service-oriented architecture

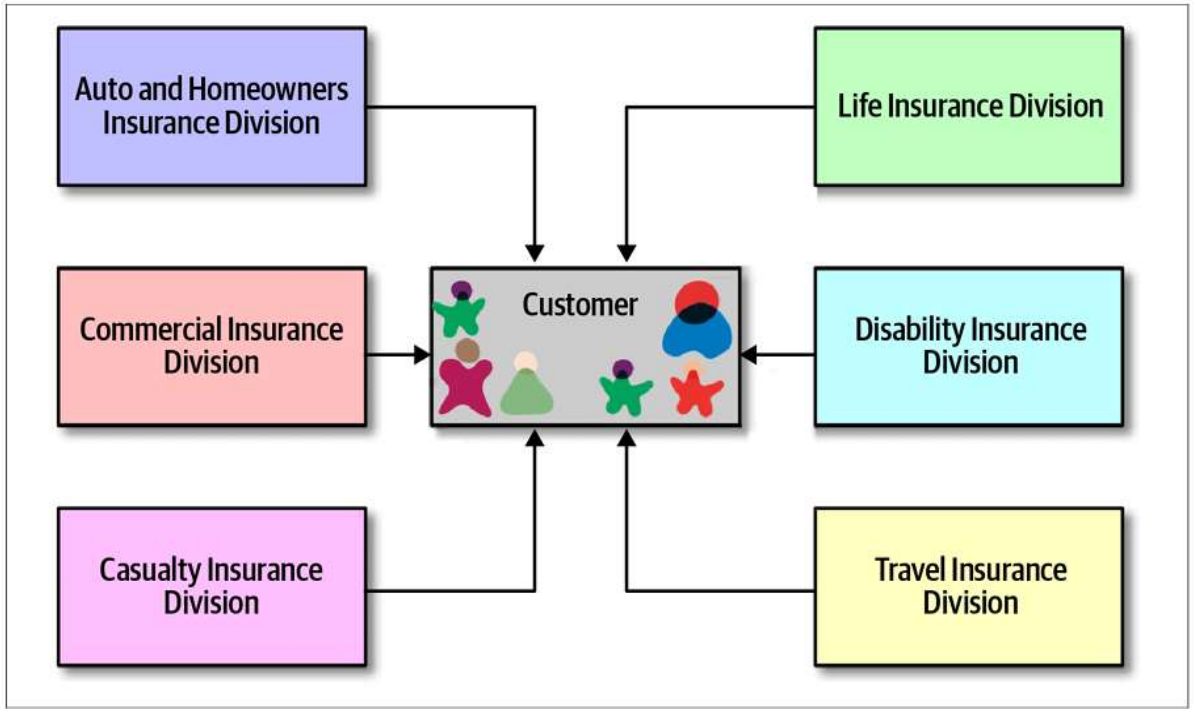
7. Orchestration-Driven Service-Oriented Architecture

Reuse…and Coupling

- A major goal of this architecture is reuse at the service level—the ability to gradually build business behavior that can be incrementally reused over time.



Seeking reuse opportunities in service-oriented architecture



Building canonical representations in service-oriented architecture

7. Orchestration-Driven Service-Oriented Architecture

Reuse…and Coupling with Example

- The proper strategy for service-oriented architecture entails extracting the customer parts into a reusable services and allowing the original services to reference the canonical Customer service.
- By isolating all customer behavior into a single Customer service, leads achieving obvious reuse goals.

Challenges in this Example

- The reuse, they also incur a huge amount of coupling between components in such a way that a change to the Customer service ripples out to all the other services, making change risky.
- The Customer service will have to include details about driver's licenses that the disability insurance division cares nothing about.

7. Orchestration-Driven Service-Oriented Architecture

Scenario: A customer places an order on the website.

Flow:

1. Customer places an order through the "Place Order" service (BS).
2. The ESB receives the request and its orchestration engine starts coordinating.
3. The ESB calls the "Inventory Service" (ES) to check stock.
4. The ESB invokes the "Payment Service" (ES) to process the payment.
5. Once payment is confirmed, the ESB calls the "Shipping Service" (ES) to arrange delivery.
6. The ESB might update the customer's profile and send a confirmation email using the "Customer Service" (ES) and "Notification Service" (ES).

- The "Inventory Service" might be built on top of an existing inventory management application (AS) and use a database service (IS) to store and retrieve data.
- The "Payment Service" might integrate with a third-party payment gateway (AS) and use security services (IS) to encrypt sensitive data.

7. Orchestration-Driven Service-Oriented Architecture

Applications

- Supply chain management systems: Orchestrate the flow of goods and services across multiple suppliers, manufacturers, and distributors.
- Healthcare systems: Orchestrate the delivery of patient care across multiple healthcare providers, hospitals, and clinics.
- Financial services systems: Orchestrate the delivery of financial products and services across multiple banks, investment firms, and insurance companies.
- Logistics and transportation systems: Orchestrate the movement of goods and services across multiple carriers, warehouses, and distribution centers

7. Orchestration-Driven Service-Oriented Architecture

Applications…

- Customer relationship management systems: Orchestrate the delivery of personalized services and products across multiple channels, such as email, social media, and mobile devices.
- Overall, Orchestration-Driven Service-Oriented Architecture is well-suited for applications that require the coordination of multiple services to achieve a specific business goal.
- It can be used in any application that requires the integration of multiple systems and services to deliver a seamless user experience.

Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★★★
Evolutionary	★
Fault tolerance	★★★
Modularity	★★★
Overall cost	★
Performance	★★
Reliability	★★
Scalability	★★★★
Simplicity	★
Testability	★

Architecture Styles

8. Micro Services Architecture

- Microservice is extremely popular architecture style.
- To understand it, we need to see the traditional Monolithic Architecture first.
- In Monolithic Architecture, usually all things are developed in one application including UI components, Business Logic, and Data Access Layer. And usually that application is deployed on one server node.
- However, microservice architecture split Business Logic and Data Access Layer into several small part called Microservices.
- Real business logic and Data access are implemented by invoking multiple different microservices distributed on different nodes

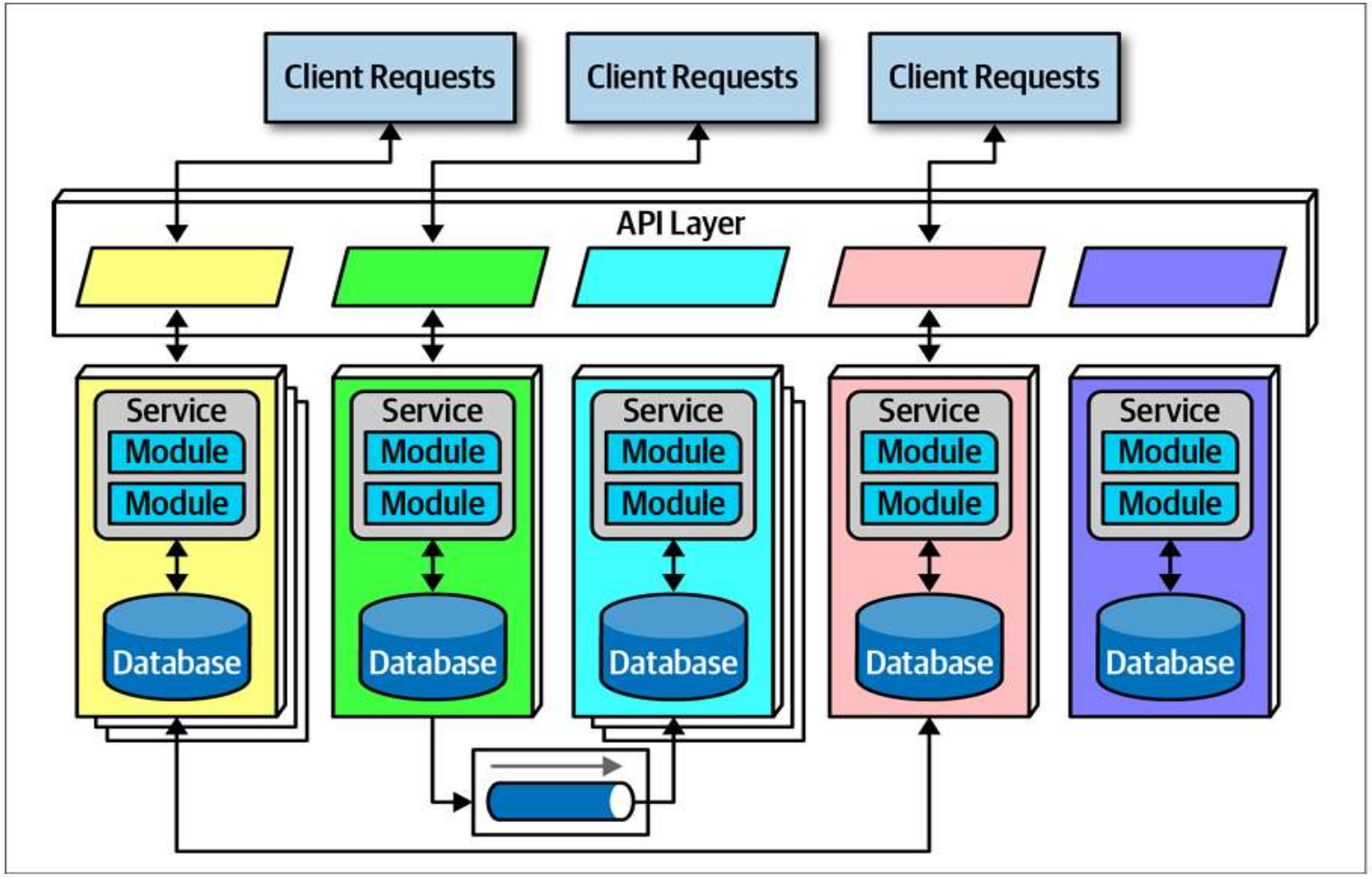
Architecture Styles

8. Micro Services Architecture

- Decompose monolithic business systems into a number of independently deployable services;
  - Each service represents a business capability, or the smallest, "atomic" unit of service that delivers business value.
- Enables speed, safety, and scale via decoupling business domain into independently deployable "bounded contexts“ (represents a decoupling style) of capabilities;
  - Each service is deployed independently (a self-service paradigm);
  - Offers independent, and efficient scaling of services.
  - Incremental change is facilitated allowing incremental deployment of new functionality.

8. Micro Services Architecture

Topology



The topology of the microservices architecture style

Architecture Styles

8. Micro Services Architecture

Distributed

- Micro services form a distributed architecture, each service runs in its own process.
- Decoupling the services to this degree allows for a simple solution to a common problem in architectures.
- Separating each service into its own process solves all the problems brought on by sharing.
- With cloud resources and container technology, micro services reap the benefits of extreme decoupling, both at the domain and operational level.
- Network calls which take much longer than method calls, and security verification at every endpoint adds additional processing time
- Service granularity such as transaction boundaries in distributed environment can become bottleneck in micro service architectures.

8. Micro Services Architecture

Bounded Context

- Micro services is the notion of bounded context, each service models a domain or workflow.
- Thus, each service includes everything necessary to operate within the application, including classes, other subcomponents, and database schemas.
- Instead of sharing of common classes as in monolithic architecture, micro services try to avoid coupling.
- Micro services take the concept of a domain-partitioned architecture to the extreme.
- Each service is meant to represent a domain or subdomain.

Granularity

- The purpose of service boundaries in micro services is to capture a domain or work flow.

Architecture Styles

8. Micro Services Architecture

Granularity Guidelines to help find the appropriate boundaries:

Purpose

- The most obvious boundary relies on the inspiration for the architecture style, a domain. Ideally, each micro service should be extremely functionally cohesive, contributing one significant behavior on behalf of the overall application.

Transactions

- Bounded contexts are business workflows, and often the entities that need to cooperate in a transaction show architects a good service boundary. Because transactions cause issues in distributed architectures. *A single "Place Order" transaction , breaking it into smaller, independent transactions (e.g., "Create Order," "Process Payment," "Update Inventory").*

Choreography : services interact directly with each other without a central orchestrator

- If an architect builds a set of services that offer excellent domain isolation yet require extensive communication to function, the architect may consider bundling these services back into a larger service to avoid the communication overhead.

8. Micro Services Architecture

Data Isolation

- Many other architecture styles use a single database for persistence.
- However, micro services tries to avoid all kinds of coupling, including shared schemas and databases used as integration points.

API Layer

- Micro services are firmly domain partitioned; they don’t need broker or mediator between the service applications.
- The API layer is often used to host service discovery, allowing a single place for user interfaces or other calling systems to find and create services in an elastic, consistent way

Architecture Styles

8. Micro Services Architecture

- Microservices architecture breaks down a *large application into smaller, independent services that can be developed, deployed, and scaled independently.*

Microservices architecture can be used in the following domains as a separate services :

- E-commerce platforms:

  Product catalog, shopping cart, payment processing, and order management.

- Social media platforms

  To handle different functions such as user authentication, profile management, messaging, and content delivery.

- Banking and financial services

  That handle different functions such as account management, transaction processing, and fraud detection.

Architecture Styles

8. Micro Services Architecture

- Healthcare applications:
  To handle patient management, electronic health records, and medical billing.
- Travel and hospitality applications:
  such as booking management, payment processing, and customer service as separate services

Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★
Fault tolerance	★★★★
Modularity	★★★★★
Overall cost	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★

Thanks !